HookLens 기획서

웹훅을 고유 URL로 받아서 실시간으로 보여주고, 정해 둔 응답을 돌려주는 개발자용 웹훅 디버거입니다.

GITHUB https://github.com/songjeongseon1/hooklens

- 작성: 2026.10.13 (개발 시작 전), 2026.10.24 결과 반영
- 기간: 1차 MVP 2주 (실제 10일, 2026.10.13 ~ 10.22)
- 인원: 1인 (기획, 백엔드, 프론트엔드, 배포 구성)

1. 배경

결제(토스페이먼츠 등)나 깃허브 연동을 만들 때는 외부 서비스가 우리 서버로 웹훅을 보내 주고, 우리는 그걸 받아서 처리하는 코드를 짭니다. 이 작업을 하면서 반복해서 막혔던 지점이 세 가지 있었습니다.

문제	지금 하는 방법	불편한 점
실제로 어떤 헤더와 바디가 오는지 모른다	문서를 읽고 추측, 또는 서버에 로그를 찍어 배포	문서와 실제가 다를 때가 있고, 로그를 보려고 매번 배포해야 함
외부 서비스가 내 PC(localhost)로 보내 주지 않는다	ngrok 같은 터널 도구를 띄움	도구를 따로 설치·실행해야 하고, 받는 서버 코드도 먼저 있어야 함
우리 서버가 실패하거나 느릴 때의 동작을 확인하기 어렵다	서버 코드에 일부러 에러를 넣음	코드를 고치고 되돌리는 일이 반복됨

2. 목표

> 가입 없이 버튼 한 번으로 웹훅 받을 주소를 만들고, 들어온 요청을 가공 없이 그대로 실시간으로 보여주며, 응답 코드와 지연을 바꿔 실패 상황을 재현할 수 있게 한다.

- 1차(2주): 위 목표의 핵심 흐름(P0 8개)을 완성하고 서버 한 대에 올릴 수 있는 배포 구성까지 만든다.
- 2차: 회원, 엔드포인트 관리, 요청 제한, 리플레이 등 계속 쓰기 위한 기능.

비슷한 도구로 webhook.site, RequestBin이 있습니다. 직접 만든 이유는 실시간 전송(SSE), 트랜잭션과 알림 순서, 커서 페이지네이션, 접근 제어처럼 백엔드에서 신경 써야 할 부분이 작은 범위 안에 다 들어 있어서입니다.

3. 사용자와 사용 상황

대상: 외부 서비스의 웹훅을 연동하는 백엔드 개발자

사용 상황 예시 — 토스페이먼츠 결제 상태 변경 웹훅 연동

1. HookLens에서 "토스 결제 웹훅 테스트"라는 이름으로 주소를 만든다.
2. 토스 개발자센터의 웹훅 URL에 그 주소를 넣고 테스트 결제를 한다.

3. HookLens 화면에 요청이 바로 뜬다. 헤더(`User-Agent`, 재전송 횟수 헤더 등)와 JSON 바디를 확인하고, 그 모양대로 DTO를 만든다.
4. 응답 설정을 `503` + 3초 지연으로 바꾼다. 토스가 실패로 보고 재전송하는지, 재전송 간격이 어떤지 목록에서 확인한다.

4. 사용자 흐름

```
[홈] 이름 입력(선택) → 엔드포인트 만들기
   │   ↳ 주소와 관리 토큰 발급. 토큰은 브라우저(localStorage)에 저장
   ▼
[엔드포인트 화면] 주소 복사 → 외부 서비스에 등록
   │
   ├─ 요청 탭: 새 웹훅이 오면 목록 맨 위에 바로 추가 → 클릭하면 오른쪽에 상세(쿼리·헤더·바디)
   │
   └─ 응답 설정 탭: 상태 코드·헤더·바디·지연 입력 → 저장 → 다음 웹훅부터 이 응답이 나감
```

같은 브라우저에서 다시 홈에 들어오면 "내 엔드포인트" 목록에서 이어서 볼 수 있습니다. 회원 기능 전이라 다른 기기에서는 볼 수 없습니다(2차에서 회원으로 이전, REQ-EP-005).

5. 요구사항

우선순위: P0 = 1차 MVP, P1 = 2차, P2 = 이후

ID	내용	우선순위	상태
REQ-EP-001	비회원도 고유 slug URL 발급	P0	완료
REQ-EP-002	비회원 엔드포인트 24시간 후 만료, 이후 수신 시 410	P0	완료
REQ-HOOK-001	`/h/{slug}/**` 모든 메서드 수신, 메서드·경로·쿼리·헤더·바디·IP·시각 저장	P0	완료
REQ-HOOK-002	설정된 응답 반환, 없으면 200 `{"ok":true}`	P0	완료
REQ-LOG-001	SSE로 새 요청 실시간 반영	P0	완료
REQ-LOG-002	요청 목록 최신순 커서 조회	P0	완료
REQ-LOG-003	요청 상세(헤더·쿼리·바디)	P0	완료
REQ-RES-001	기본 응답 설정(상태 코드, 헤더, 바디, 지연)	P0	완료
REQ-EP-003~005	내 엔드포인트 목록·수정·삭제, 비회원 엔드포인트를 회원으로 이전	P1	
REQ-HOOK-003	엔드포인트당 분당 60건 초과 시 429	P1	
REQ-HOOK-004	바디 1MB 초과 시 413	P1	
REQ-LOG-004~005	로그 비우기, 매일 03:00 보관 기간 지난 로그 삭제	P1	
REQ-RPL-001~002	받은 요청을 다른 주소로 재전송, 내부망 주소 차단(SSRF)	P1	
REQ-AUTH-001~002	회원가입·로그인(JWT), 토큰 재발급·로그아웃	P1	
REQ-RES-002	조건부 응답 (메서드·헤더·바디 내용에 따라 다른 응답)	P2	

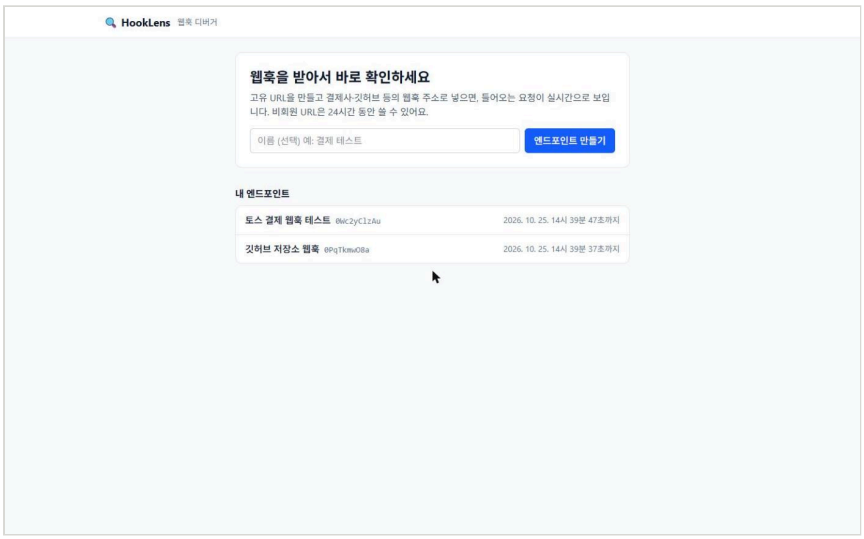
ID	내용	우선순위	상태
REQ-RPL-003	자동 포워딩 (받은 요청을 내 로컬 서버로 바로 전달)	P2	
REQ-UX-001~002	cURL 명령어로 내보내기, 요청 검색	P2	

범위를 이렇게 나눈 기준: "주소 만들기 → 받기 → 보기 → 응답 바꾸기"가 끊김 없이 되는 것까지를 1차로 잡았습니다. 회원과 요청 제한은 계속 쓰려면 필요하지만, 없어도 핵심 흐름은 확인할 수 있어서 2차로 미뤘습니다.

6. 화면 설계

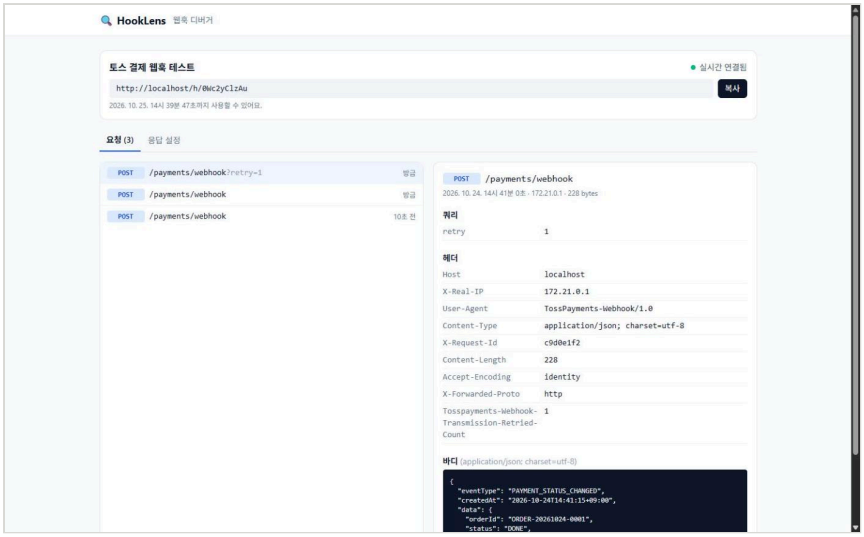
화면은 두 개입니다.

홈 `/`

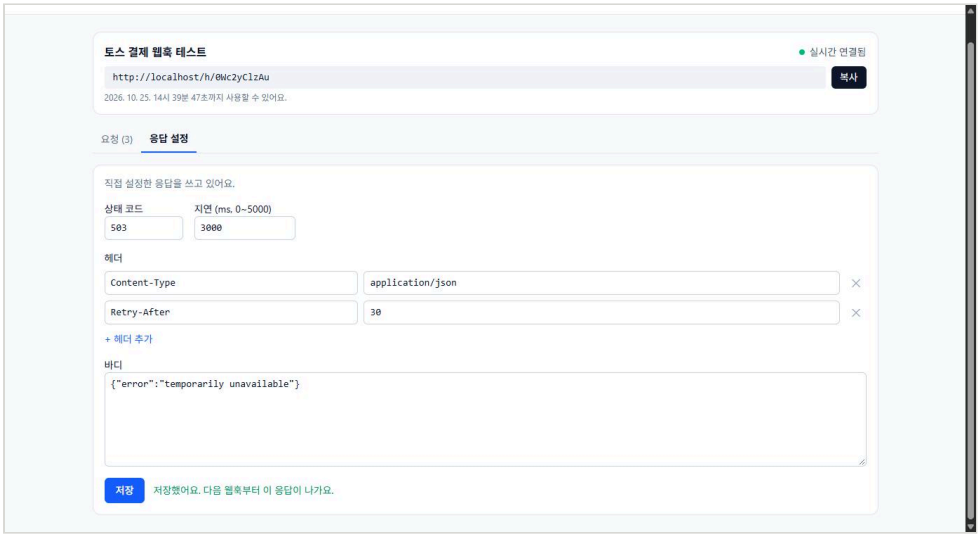


영역	내용
발급	이름(선택, 50자) 입력 + "엔드포인트 만들기" 버튼. 만들면 바로 엔드포인트 화면으로 이동
내 엔드포인트	이 브라우저에서 만든 엔드포인트 목록 (이름, slug, 만료 시각). 클릭하면 이동

엔드포인트 `/e/{slug}`



영역	내용
상단	이름, 웹훅 주소 + 복사 버튼, 만료 시각, 실시간 연결 상태(연결 중 / 실시간 연결됨 / 다시 연결 중 / 연결 실패)
요청 탭 (왼쪽)	요청 목록 최신순. 메서드 배지, 경로, "3분 전" 형식 시각. 새 요청은 새로고침 없이 맨 위에 추가. 맨 아래 "더 보기"로 다음 페이지
요청 탭 (오른쪽)	선택한 요청의 상세. 메서드·경로·전체 시각·IP·크기, 쿼리 표, 헤더 표, 바디(JSON이면 들여쓰기 정렬)



영역	내용
응답 설정 탭	상태 코드, 지연(ms), 헤더(이름·값 줄 추가·삭제), 바디. 저장하면 "다음 웹훅부터 이 응답이 나가요" 표시. 설정 전에는 기본 응답(200, `{"ok":true}`)이 채워져 있음

7. 정책

항목	정책	이유
웹훅 주소	`/h/{slug}`, slug는 영숫자 10자 랜덤	추측하기 어렵고 URL에 넣기 쉬움
비회원 만료	발급 후 24시간. 이후 수신·조회 모두 410	가입 없이 쓰는 만큼 데이터가 무한히 쌓이지 않게
관리 권한	발급 응답에서 토큰(랜덤 32바이트)을 한 번만 제공, 서버는 SHA-256 해시만 저장	DB가 유출돼도 토큰 원문은 알 수 없음
남의 요청 조회	404로 응답 (403 아님)	존재 여부도 알려 주지 않기 위해
응답 상태 코드	100~599	HTTP 상태 코드 범위
응답 지연	0~5초	너무 길면 보낸 쪽이 타임아웃으로 끊음
응답 헤더	20개까지, 이름은 HTTP 토큰 문자만, 값에 줄바꿈 불가	헤더 주입 방지
응답 바디	20,000자까지	DB TEXT(64KB) 안에 한글로 채워도 들어가게
요청 바디 크기	1MB 초과 시 413 (2차)	1차는 Nginx에서 2MB로 제한
요청 빈도	엔드포인트당 분당 60건 초과 시 429 (2차, Redis)	무한 재전송 등으로 DB가 차는 것 방지
보관 기간	회원 로그 30일, 만료된 비회원 엔드포인트는 통째로 삭제. 매일 03:00, 1,000건씩 나눠 삭제 (2차)	한 번에 지우면 테이블 잠금이 길어짐
시간	모든 계층 KST, API는 `+09:00` 오프셋 포함	서버(UTC)와 화면 사이 9시간 어긋남 방지

8. 데이터 모델

1차에서 쓰는 테이블은 세 개입니다. 엔드포인트를 지우면 요청 로그와 응답 규칙도 함께 지워집니다(`on delete cascade`).

```
endpoint 1 ─── N request_log
    │
    └──────── 1 ── N response_rule   (1차는 기본 규칙 1개만 사용)
```

테이블	주요 컬럼	비고
endpoint	endpoint_id, slug(UK), name, anonymous_token_hash, expires_at, member_id, created_at	member_id null = 비회원 (member 테이블은 2차)
request_log	request_id, endpoint_id(FK), method, path, query_string, headers(JSON), body, content_type, ip, size_bytes, received_at	인덱스 `(endpoint_id, request_id DESC)` — 최신순 커서 조회용

테이블	주요 컬럼	비고
response_rule	rule_id, endpoint_id(FK), is_default, status_code, response_headers(JSON), response_body, delay_ms	priority·match_* 컬럼은 조건부 응답(P2) 때 추가

API의 요청·응답 형식은 API 명세서에 있습니다.

9. 기술 선택

항목	선택	대안	고른 이유
실시간 전송	SSE	WebSocket, 폴링	서버 → 화면 한 방향 알림이라 충분. HTTP 그대로라 Nginx·인증 처리가 단순
SSE 인증	fetch 스트림 + 헤더	EventSource + 쿼리 파라미터	토큰이 URL(접근 로그, 브라우저 기록)에 남지 않음. 관리 API와 인증 방식 하나로 통일
페이지네이션	커서	오프셋	새 요청이 계속 들어오는 목록이라 오프셋은 중복·누락이 생김
통합 테스트	Testcontainers (실제 MySQL)	H2	JSON 컬럼, 인덱스, 시간대가 운영과 같은 조건에서 검증됨
배포	EC2 한 대 + Docker Compose	ECS, RDS	1차 규모에 맞게 단순하게. DB는 접속 주소만 바꾸면 RDS로 옮길 수 있게 구성

10. 일정 — 계획과 실제

	계획	실제
1주차 (10/13~17)	세팅, 발급(EP-001·002), 수신·저장(HOOK-001·002), 목록·상세 API(LOG-002·003)	10/13 세팅·발급, 10/14 수신·저장, 10/15 응답 반환. 목록·상세는 다음 주 월요일(10/19)로 밀림
2주차 (10/19~23)	SSE(LOG-001), 응답 설정(RES-001), React 화면, EC2 배포	10/19 목록·상세, 10/20 SSE, 10/21 응답 설정, 10/22 React 화면·배포 구성. EC2 실제 배포는 보류

- 결과: P0 8개 완료, 테스트 59개, 커밋 26개, PR 8개 (10일)
- 다음에 바꿀 점: 배포 구성을 마지막이 아니라 초반에 먼저 만들어 두고 기능을 올리는 순서로 하려고 합니다.